

Lab Report

Course Code: CSE-4742
Course Title: Computer Graphics Lab

Autumn 2026
Section: 7BF

Lab No: 2

Name of Labwork: Implementation of Bresenham's Line Drawing Algorithm

Student's ID : C233456
Date of Performance : 06-09-2026
Date of Submission : 05-09-2026

Marks : _____

1. Number & Name of the Experiment

Lab 2: Implementation of Bresenham's Line Drawing Algorithm

2. Introduction

Bresenham's line drawing algorithm is an efficient rasterization algorithm used in computer graphics to draw a straight line between two given points. It determines the most suitable pixel at each step so that the plotted pixels closely represent the required mathematical line.

Unlike the DDA algorithm, Bresenham's algorithm uses integer-based calculations and a decision parameter instead of floating-point calculations. This makes it faster and reduces the computational cost of rounding operations.

The algorithm considers two cases depending on the direction in which the line changes more: $|dx| \geq |dy|$ and $|dx| < |dy|$. In each case, one coordinate is increased at every iteration, while the decision parameter determines whether the other coordinate should also be changed.

In this implementation, the starting and ending coordinates are first taken as (x_1, y_1) and (x_2, y_2) . The absolute differences dx and dy are calculated, along with the movement directions for x and y . The appropriate case is then selected, and the line is generated using a loop with an **if-else** decision at each step.

Because it uses only simple integer arithmetic and avoids floating-point operations, Bresenham's algorithm is more efficient than DDA and is widely used for raster-based line drawing.

3. Algorithm

The Bresenham line drawing algorithm is implemented as follows:

1. Take the starting point (x_1, y_1) and ending point (x_2, y_2) .
2. Calculate the absolute coordinate differences:

$$dx = |x_2 - x_1|$$

$$dy = |y_2 - y_1|$$

3. Determine the direction of movement:

$$stepX = \begin{cases} 1, & x_1 < x_2 \\ -1, & x_1 \geq x_2 \end{cases} \quad stepY = \begin{cases} 1, & y_1 < y_2 \\ -1, & y_1 \geq y_2 \end{cases}$$

4. Compare dx and dy to select the driving coordinate.
5. If $dx \geq dy$, use x as the driving coordinate and initialize:

$$d_s = 2dy$$

$$d_t = 2(dy - dx)$$

$$d = 2dy - dx$$

6. Start from (x_1, y_1) and plot the first pixel. Run a loop until $x = x_2$. At each iteration, increase x by **stepX**.
7. Use an **if-else** condition on d :

- If $d < 0$, update

$$d = d + d_s$$

without changing y .

- Otherwise, update y by **stepY** and use

$$d = d + d_t$$

8. If $dx < dy$, use y as the driving coordinate and initialize:

$$d_s = 2dx$$

$$d_t = 2(dx - dy)$$

$$d = 2dx - dy$$

9. Start from (x_1, y_1) and plot the first pixel. Run a loop until $y = y_2$. At each iteration, increase y by **stepY**.
10. Again, use an **if-else** condition on d :
 - If $d < 0$, update
$$d = d + d_s$$

without changing x .

 - Otherwise, update x by **stepX** and use
$$d = d + d_t$$
11. Plot each calculated pixel using **putpixel()** until the ending coordinate is reached.

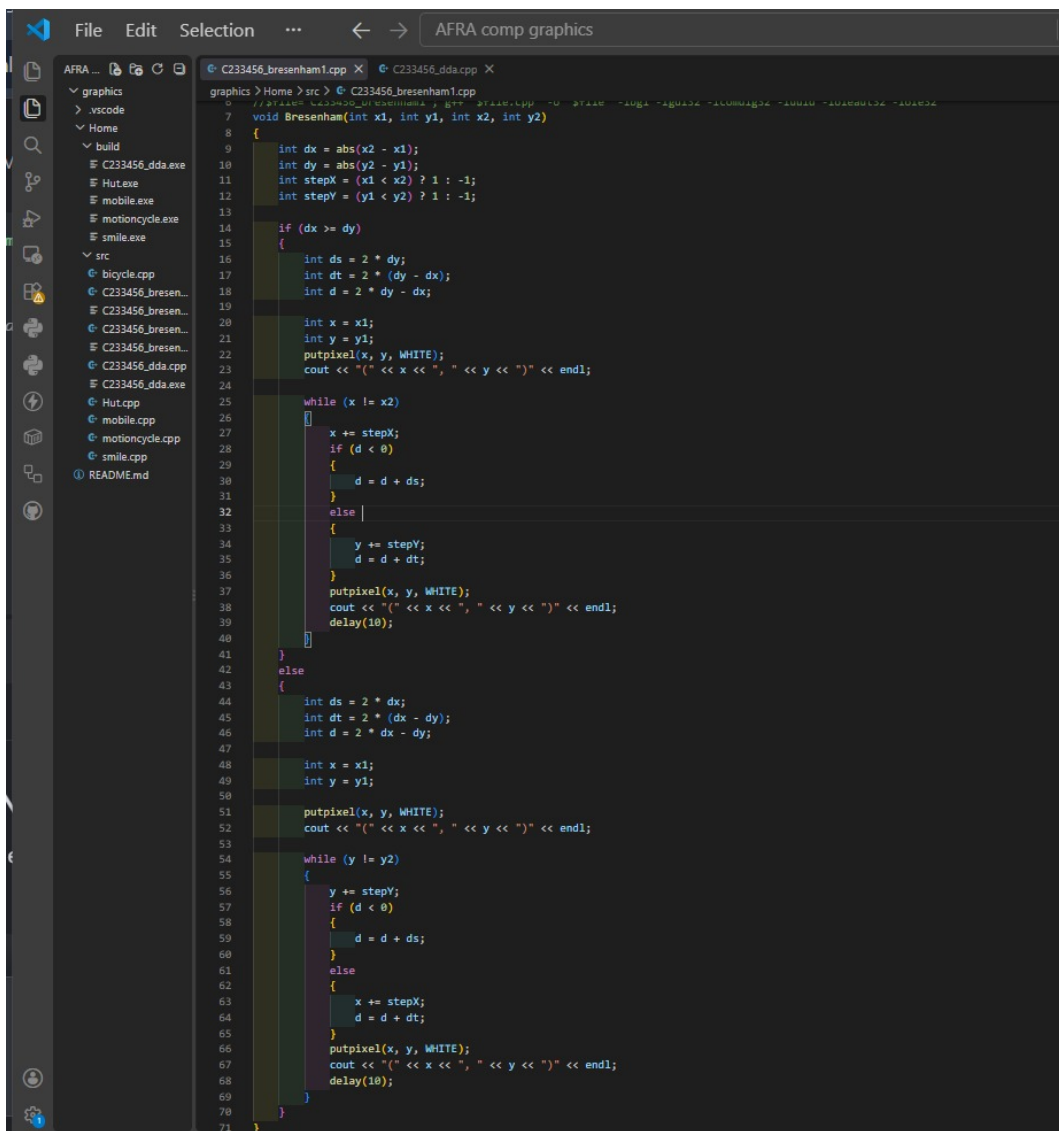
The program implements the algorithm in the `Bresenham()` function. It calculates dx , dy , `stepX`, and `stepY`, then selects the appropriate $dx \geq dy$ or $dx < dy$ case. The decision parameters d , d_s , and d_t are updated inside a `while` loop using an `if-else` condition to select the next pixel. The pixels are plotted using `putpixel()`, while `cout` displays the coordinates and `delay(10)` makes the drawing process visible. .

4. Program with Screenshot

The Bresenham line drawing algorithm was implemented in C++ using Visual Studio Code with the graphics library.

Code Screenshot

The following screenshot shows the complete implementation:



```

1 // BRESENHAM LINE DRAWING ALGORITHM
2
3 void Bresenham(int x1, int y1, int x2, int y2)
4 {
5     int dx = abs(x2 - x1);
6     int dy = abs(y2 - y1);
7     int stepX = (x1 < x2) ? 1 : -1;
8     int stepY = (y1 < y2) ? 1 : -1;
9
10    if (dx >= dy)
11    {
12        int ds = 2 * dy;
13        int dt = 2 * (dy - dx);
14        int d = 2 * dy - dx;
15
16        int x = x1;
17        int y = y1;
18        putpixel(x, y, WHITE);
19        cout << "(" << x << ", " << y << ")" << endl;
20
21        while (x != x2)
22        {
23            x += stepX;
24            if (d < 0)
25            {
26                d = d + ds;
27            }
28            else
29            {
30                y += stepY;
31                d = d + dt;
32            }
33            putpixel(x, y, WHITE);
34            cout << "(" << x << ", " << y << ")" << endl;
35            delay(10);
36        }
37    }
38    else
39    {
40        int ds = 2 * dx;
41        int dt = 2 * (dx - dy);
42        int d = 2 * dx - dy;
43
44        int x = x1;
45        int y = y1;
46        putpixel(x, y, WHITE);
47        cout << "(" << x << ", " << y << ")" << endl;
48
49        while (y != y2)
50        {
51            y += stepY;
52            if (d < 0)
53            {
54                d = d + ds;
55            }
56            else
57            {
58                x += stepX;
59                d = d + dt;
60            }
61            putpixel(x, y, WHITE);
62            cout << "(" << x << ", " << y << ")" << endl;
63            delay(10);
64        }
65    }
66 }

```

Figure 1: Bresenham line drawing program

5. Output/Result

The program was successfully executed and generated a straight line between the specified starting and ending coordinates.

The ID(C233456) was displayed in the graphics window as required.

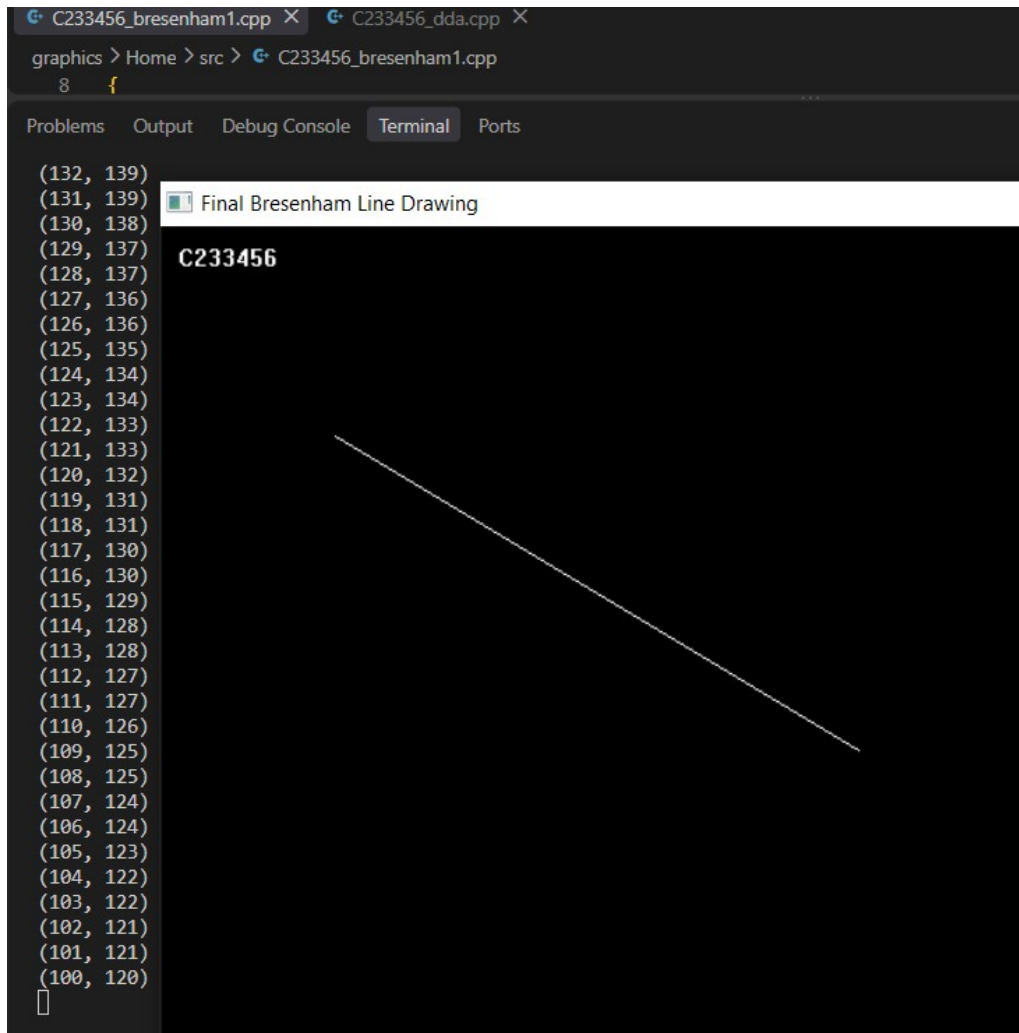


Figure 2: Output of Bresenham's line drawing algorithm from (400,300) to (100,120)

6. Discussion

Bresenham's algorithm successfully generated the required line using an integer-based decision parameter to select the next suitable pixel.

At each step, the `if-else` condition checks the decision parameter and updates the driving and non-driving coordinates accordingly. This avoids the floating-point calculations and repeated rounding required by DDA, making Bresenham's algorithm more efficient for raster line generation.

The implementation can also be adapted for different coordinate ranges, such as two-digit

coordinates, by changing the input coordinates or applying suitable scaling. Different input coordinates were used to observe the resulting line.

7. Conclusion

Bresenham's line drawing algorithm was successfully implemented in C++ using Visual Studio Code with the graphics library. The experiment demonstrated how integer-based calculations and a decision parameter can efficiently convert a mathematical line into pixels on a raster display.

It also provided an understanding of rasterization and the difference between Bresenham's algorithm and the DDA algorithm. The program can be modified for different coordinate ranges by changing the input or applying scaling.